

HealthCare Claims Data Pipeline

Q. Build a scalable ETL pipeline that processes healthcare claims data and generates analytics dashboards/tables.

1. Architecture

- RAW CSV Data
- Bronze Layer (RAW Ingestion)
- Silver Layer (Data Cleaning)
- Gold Layer (Business Analytics, Creating business ready datasets using sql)
- Dashboards -> Analytics Reports/ SQL Queries

2. Dataset

<https://www.cms.gov/data-research/statistics-trends-and-reports/medicare-claims-synthetic-public-use-files/cms-2008-2010-data-entrepreneurs-synthetic-public-use-file-de-synpuf/de10-sample-4>

3. Understanding the dataset

1. Data dictionary codebook - Understanding different columns in the dataset.

BENE - Beneficiary, DT - Date, CD - Code, IND - Indicator, CVRGE - Coverage, MONS - Months, IP - Inpatient, OP - Outpatient, CAR - Carrier/physician claims, MEDREIMB - Medicare reimbursement, BENERES - Beneficiary Responsibility, PPPYMT - Primary Payer Payment, SP - Special Condition/chronic

DESYNPUF_ID - BeneficiaryId, BENE_BIRTH_DT- Birthdate, BENE_DEATH_DT- Death date, BENE_SEX_IDENT_CD-Gender code, BENE_RACE_CD - Race code, BENE_ESRD_IND-Endstage renal disease indicator, SP_STATE_CODE-state code, BENE_COUNTY_CD-country code, BENE_HI_CVRAGE_TOT_MONS-hospital coverage total months,

BENE_SMI_CVRAGE_TOT_MONS - Medical Insurance Coverage months,
BENE_HMO_CVRAGE_TOT_MONS - HMO Coverage months,
PLAN_CVRG_MOS_NUM - Plan Coverage months, SP_ALZHDMTA-
Alzhiemers or Dementia Indicator, SP_CHF- Congestive Heart
failure indicator, SP_CHRNKIDN- Chronic Kidney Disease
Indicator, SP_CNCR-Cancer Indicator, SP_COPD-COPD Indicator,
SP_DEPRESSN-Depression Indicator, SP_DIABETES-Diabetes
Indicator, SP_ISCHMCHT-Ischemic Heart disease indicator,
SP_OSTEOPRS- OsteoporosisIndicator, SP_RA_OA-Arthritis,
SP_STRKETIA - Stroke Indicator, MEDREIMB_IP-Medical
Reimbursement for inpatient, BENRES_IP- Beneficiary
Responsibility Inpatient, PPPYMT_IP - PrimaryPayerPayment
Inpatient, MEDREIMB_OP-Medicare imburement op, BENRES_OP-
beneficiary respos

1. Bronze Layer

3.1 Installation and Setup

Shell

```
!apt-get install openjdk-11-jdk -y
!pip install pyspark
import os
os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-11-openjdk-amd64"
import os
print("JAVA_HOME:", os.environ.get("JAVA_HOME"))
!java -version
!pip install --upgrade pyspark
```

3.2 Start the Spark Session

Python

```
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .master("local[*]") \
    .appName("CMS Project") \
    .getOrCreate()
```

```
print("Spark started")
```

Spark Session started. This is like the gateway to all the analysis and the further part

3.3 Load the file from local

Python

```
# Load the beneficiary file
from google.colab import files
uploaded = files.upload()
beneficiarydf = spark.read.csv(
    "DE1_0_2008_Beneficiary_Summary_File_Sample_4.csv",
    header=True,
    inferSchema=True
)
```

And

Python

```
# Load the beneficiary file
from google.colab import files
uploaded = files.upload()
inpatientdf = spark.read.csv(
    "DE1_0_2008_to_2010_Inpatient_Claims_Sample_4.csv",
    header=True,
    inferSchema=True
)
```

3.4 Compare the results

'DESYNPUF_ID' -> present in both the files.

-----Bronze layer is complete now-----

Step 2: Silver Layer

Silver Layer = cleaning the data and understanding the structure

2.1 Data Understanding

Beneficiary file has all patient data and Inpatient are the ones currently undergoing some treatment

2.2.1 3 core entities

1. Patient -> Beneficiary file -> Patient distribution, patient has chronic disease or no, insurance coverage
2. Inpatient -> Stays in the hospital
3. Outpatient -> not in the hospital

2.2.2 Files are connected using the DESYNPUF_ID.

Python

```
beneficiarydf.groupBy("BENE_SEX_IDENT_CD").count().show()
beneficiarydf.groupBy("SP_STATE_CODE").count().show()
beneficiarydf.groupBy("SP_ALZHDMTA").count().show()
beneficiarydf.filter(df["SP_DIABETES"] == 1).count()
```

Importing the claims data into the environment

Python

```
from google.colab import files
uploaded = files.upload()
```

Python

```
inpatientdf = spark.read.csv(
    "DE1_0_2008_to_2010_Inpatient_Claims_Sample_4.csv",
    header=True,
    inferSchema=True
)
```

Reviewing the column structure and calculating summary statistics

```
Python
inpatientdf.columns
inpatientdf.describe().show()
```

Determining the total number of unique patient IDs

```
Python
inpatientdf.select("DESYNPUF_ID").distinct().count()
```

2.1 Calculating total cost

Import the col function to handle column operations

Calculate the aggregate medical reimbursement cost

```
Python
from pyspark.sql.functions import col
benecostdf = df.withColumn(
    "TOTAL_COST",
    col("MEDREIMB_IP") + col("MEDREIMB_OP") + col("MEDREIMB_CAR")
)
```

2.2 Joining Datasets

```
Python
joined_df = beneficiarydf.join(inpatientdf, on="DESYNPUF_ID",
how="inner")
```

In this we are actually joining the 2 tables beneficiary and inpatient.

2.3 Silver Transformation Layer for the joined dataset

❑ Remove duplicates / repeated columns

- ☐ Aggregate costs correctly
- ☐ Build patient level table
- ☐ NULL values
- ☐ Wrong Datatypes

1. Change str datatype to numeric for values

Python

```
# Silver Transformation Layer
from pyspark.sql.functions import col
numeric_cols = ["MEDREIMB_IP", "BENRES_IP", "PPPYMT_IP"]
for c in numeric_cols:
    benef_inpatientdf = benef_inpatientdf.withColumn(c,
col(c).cast("double"))
#2. Fill NULL values with 0
benef_inpatientdf = benef_inpatientdf.fillna({
    "MEDREIMB_IP": 0,
    "BENRES_IP": 0,
    "PPPYMT_IP": 0,
    "ICD9_DGNS_CD_4": "UNKNOWN",
    "ICD9_DGNS_CD_5": "UNKNOWN",
    "ICD9_DGNS_CD_6": "UNKNOWN",
    "ICD9_DGNS_CD_7": "UNKNOWN",
    "ICD9_DGNS_CD_8": "UNKNOWN",
    "ICD9_DGNS_CD_9": "UNKNOWN",
    "ICD9_DGNS_CD_10": "UNKNOWN",
    "ICD9_PRCDR_CD_1": "UNKNOWN",
    "ICD9_PRCDR_CD_2": "UNKNOWN",
    "ICD9_PRCDR_CD_3": "UNKNOWN",
    "ICD9_PRCDR_CD_4": "UNKNOWN",
    "ICD9_PRCDR_CD_5": "UNKNOWN",
    "ICD9_PRCDR_CD_6": "UNKNOWN",
    "HCPCS_CD_1": "UNKNOWN",
    "HCPCS_CD_2": "UNKNOWN",
    "HCPCS_CD_3": "UNKNOWN",
    "HCPCS_CD_4": "UNKNOWN"
})
```

```

}))
# 3. Drop rows with missing keys
benef_inpatientdf =
benef_inpatientdf.dropna(subset=["DESYNPUF_ID"])
# 4. drop duplicates
before = benef_inpatientdf.count()
benef_inpatientdf = benef_inpatientdf.dropDuplicates()      #
dropping duplicate values
after = benef_inpatientdf.count()
print("Removed duplicates= " + str(before - after))

#5. Convert dates
from pyspark.sql.functions import to_date
benef_inpatientdf = benef_inpatientdf.withColumn(
    "CLM_FROM_DT",
    to_date(col("CLM_FROM_DT"), "yyyyMMdd")
)
benef_inpatientdf = benef_inpatientdf.withColumn(
    "CLM_THRU_DT",
    to_date(col("CLM_THRU_DT"), "yyyyMMdd")
)
# 6. Calculating new metric called length of stay by subtracting
the clm_thru_dt to clm_from_date
from pyspark.sql.functions import datediff
benef_inpatientdf = benef_inpatientdf.withColumn(
    "LENGTH_OF_STAY",
    datediff(col("CLM_THRU_DT"), col("CLM_FROM_DT"))
)
#7. Healthcare financial records cannot be negative
benef_inpatientdf = benef_inpatientdf.filter(
    col("MEDREIMB_IP") >= 0
)
benef_inpatientdf.show()

# Standardising column names

```

```
rename_map = {  
    "DESYNPUF_ID": "patient_id",  
    "BENE_BIRTH_DT": "birth_date",  
    "BENE_DEATH_DT": "death_date",  
    "BENE_SEX_IDENT_CD": "gender_code",  
    "BENE_RACE_CD": "race_code",  
    "BENE_ESRD_IND": "esrd_indicator",  
  
    "SP_STATE_CODE": "state_code",  
    "BENE_COUNTY_CD": "county_code",  
  
    "SP_ALZHDMTA": "has_alzheimers",  
    "SP_CHF": "has_heart_failure",  
    "SP_CHRNKIDN": "has_chronic_kidney_disease",  
    "SP_CNCR": "has_cancer",  
    "SP_COPD": "has_copd",  
    "SP_DEPRESSN": "has_depression",  
    "SP_DIABETES": "has_diabetes",  
    "SP_ISCHMCHT": "has_ischemic_heart_disease",  
    "SP_OSTEOPRS": "has_osteoporosis",  
    "SP_RA_OA": "has_arthritis",  
    "SP_STRKETIA": "has_stroke",  
  
    "MEDREIMB_IP": "medicare_reimbursement_ip",  
    "BENRES_IP": "beneficiary_responsibility_ip",  
    "PPPYMT_IP": "primary_payer_payment_ip",  
  
    "CLM_ID": "claim_id",  
    "SEGMENT": "claim_segment",  
    "CLM_FROM_DT": "claim_start_date",  
    "CLM_THRU_DT": "claim_end_date",
```



```

    "PRVDR_NUM": "provider_id",

    "CLM_PMT_AMT": "claim_payment_amount",
    "CLM_UTLZTN_DAY_CNT": "utilization_days"
}

for old_name, new_name in rename_map.items():
    benef_inpatientdf = benef_inpatientdf.withColumnRenamed(old_name,
new_name)

benef_inpatientdf.show()

```

----- Silver Layer is complete-----

Step 3: GOLD layer - ETL Pipeline

KPI = Key Performance Indicator. A measurable number used to evaluate performance or answer an important business question.

Transform cleaned data into tables. Use this for dashboards, reporting, analytics or decision making etc.

Product managers or business managers want,

Question-1: Which providers generate highest healthcare cost?

1. Convert question into a measurable metric

For example, for this question it is highest healthcare cost per provider average claim cost? Or cost per patient? Etc

2. Identify the grain

What is one row in the result supposed to represent?
Patient? Provider?

3. Identify required tables

Which data contains the answer?
Total inpatient cost by provider

4. What calculations are needed?

Which providers generate highest healthcare cost?

Step 1: Understand the metric

Total medicare cost = Medicare + Primary payer payment +
Beneficiary Responsibility

Step 2: Build total cost

Step 3: Create Gold table

Python

```
# printing total cost
from pyspark.sql.functions import col

# Medicare cost
benecostdf = beneficiarydf.withColumn(
    "TOTAL_COST",
    col("medicare_reimbursement_ip") +
    col("medicare_reimbursement_op") + col("medicare_reimbursement_car")
)
benecostdf.show()
```

Python

Calculating total healthcare cost

```
from pyspark.sql.functions import col, sum
provider_df = (inpatientdf
    .withColumn(
        "total_healthcare_cost",      # Name of the new column you
are creating
        col("claim_payment_amount") +    # Value
        col("primary_payer_paid_amount") +
        col("inpatient_deductible_amount")
    )
    .groupBy("provider_id") # group by clause
    .agg(
        sum("total_healthcare_cost") # for finding aggregate
        .alias("total_healthcare_cost")
    )
)
provider_df.orderBy(
    col("total_healthcare_cost").desc() # order by desc
).show(10) # shows largest 10
```

```
+-----+-----+
|provider_id|total_healthcare_cost|
+-----+-----+
|      23006G|          9046688.0|
|      1000AH|          7426576.0|
|      2100WC|          4786016.0|
|      3600NG|          4778636.0|
|      2600ZT|          4477012.0|
|      1401RR|          4309848.0|
|      3400XN|          4066684.0|
|      4500DP|          3791236.0|
|      0501MA|          3632652.0|
|      1002DC|          3546912.0|
+-----+-----+
```

Q. Which providers handle the most patients?

1. Group by provider
2. Sum payments
3. Sort descending

Python

```
from pyspark.sql.functions import col, sum

provider_df = (
    inpatientdf
    .groupBy("provider_id")
    .agg(
        sum("claim_payment_amount").alias("total_payments")
    )
)

provider_df.orderBy(
    col("total_payments").desc()
).show(10)
```

Output

provider_id	total_payments
P1	1,200,000
P2	950,000

Q3. Which providers handle most payments

Total money paid per provider

Type of payments present,

claim_payment_amount -> Money paid by medicare. Main insurance company pays most of the bill.

Primary_payer_paid_amount -> Money paid by other insurance.

Coinsurance_liability_amount -> Patient shares a % of the bill.

inpatient_deductible_amount -> Fixed amount patient must pay first

pass_through_per_diem_amount -> Extra reimbursement paid to the hospital for special cases.

Which providers generate highest total healthcare cost?

```
SELECT
    provider_id,
    SUM(
        claim_payment_amount +
        primary_payer_paid_amount +
        inpatient_deductible_amount +
        coinsurance_liability_amount +
        blood_deductible_liability_amount +
        pass_through_per_diem_amount
    ) AS total_healthcare_cost
FROM inpatient
GROUP BY provider_id
ORDER BY total_healthcare_cost DESC;
```

Step 4: **Dashboard generation using tableau**

Python

Step 1: Export data to tableau

```
provider_gold.toPandas().to_csv(  
    "provider_gold.csv",  
    index=False  
)
```

This will create provider_dashboard.csv and once we download it, then it has the provider_df table.

None

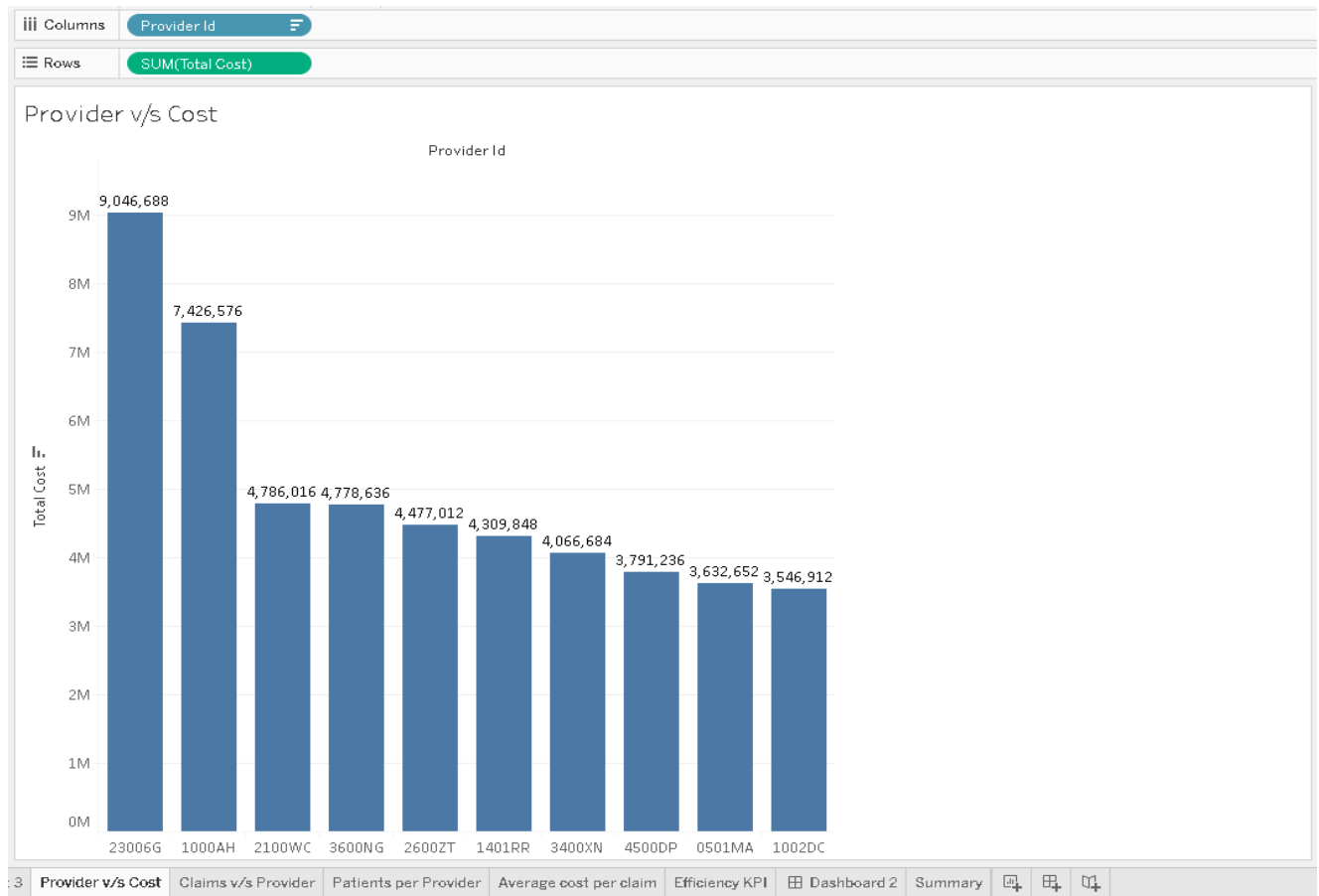
Step 2: Upload it to Tableau Web

1. Created sheets ->

Sheets:

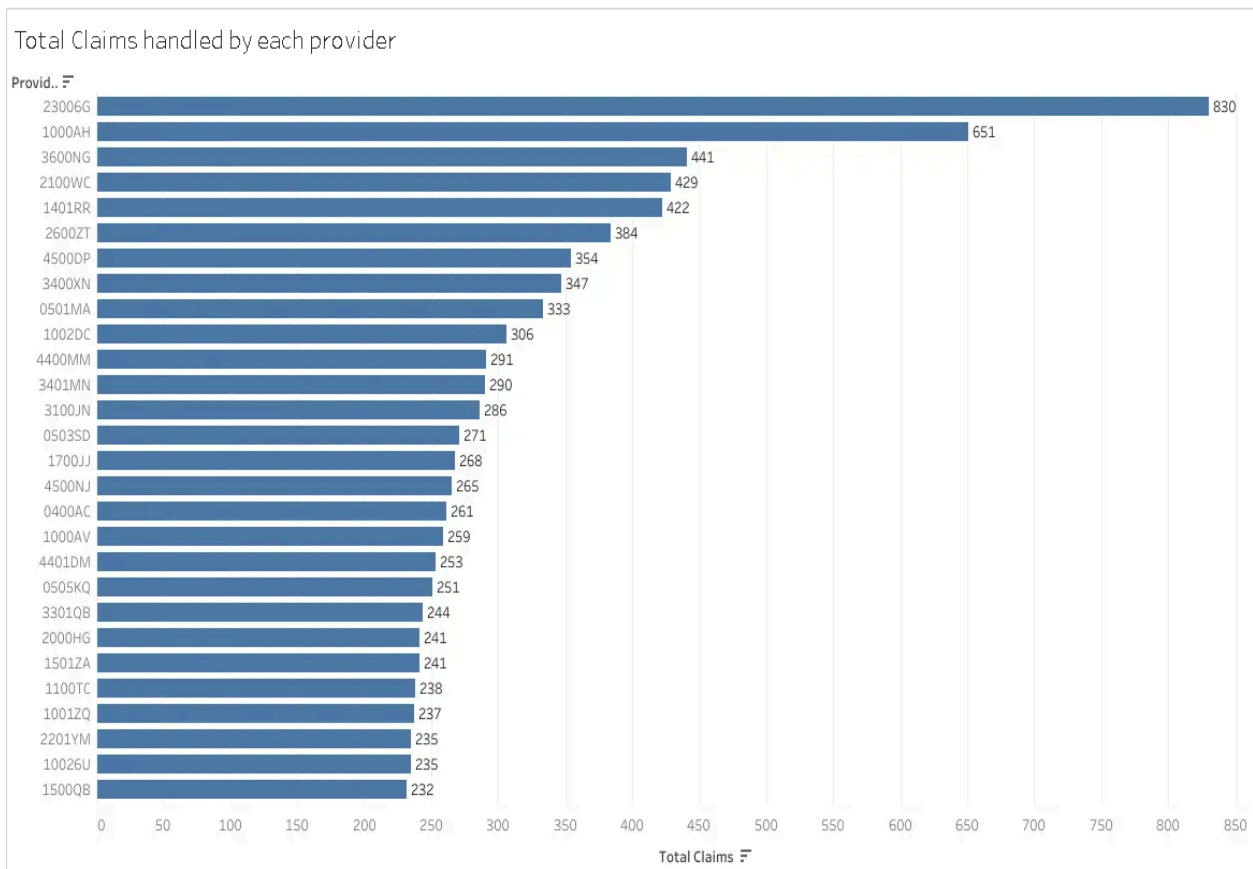
1. Provider v/s Cost:

- a. Shows the relationship between provider v/s cost. This view shows the total healthcare claim cost associated with each provider.
- b. Each bar = one healthcare provider.
Height of bar = total cost of all claims handled by that provider
Higher bar = Indicates higher overall healthcare spending/claim amount



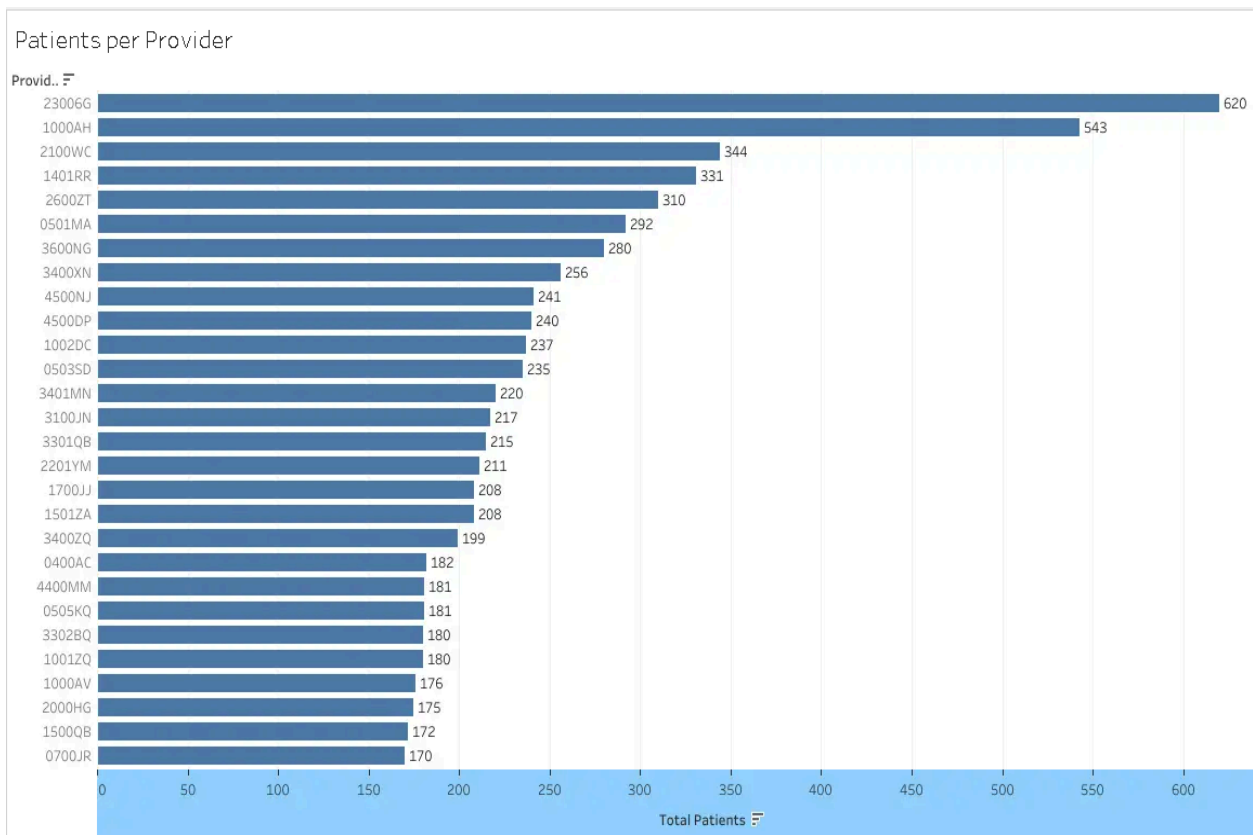
2. Claims v/s Provider:

- Total claims handled by each provider. This shows the comparison of total healthcare expenditures across providers.
- Provider **23006G** has the highest total claim cost (~9M) .



3. Patients per Provider

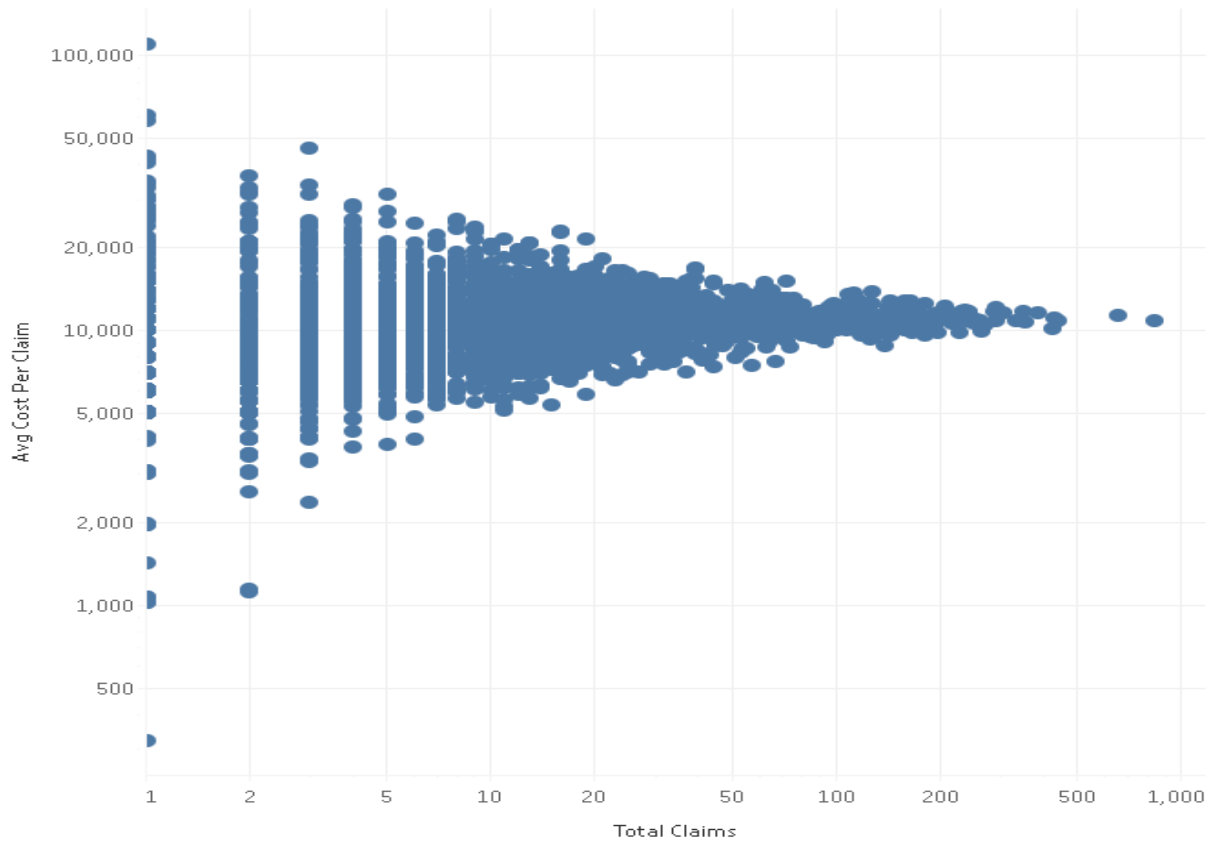
- This view describes the patients handled by each provider.
- Each horizontal bar represents one provider and length represents higher provider utilisation.
- Provider **23006G** handled the highest number of claims (~830)



3. Average cost per claim:

- This describes relationships between total claims and average cost per claim across different healthcare providers.
- Providers with less number of claims show wide range of average costs. Some are very low and some are extremely high.
- Providers with comparatively higher number of claims maintain average costs

Average cost per claim

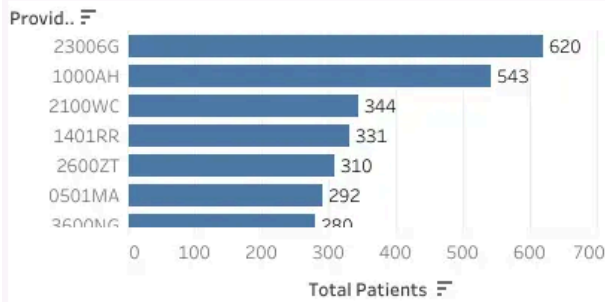


Final Summary Dashboard

Healthcare Provider Efficiency Overview

Total Cost	Total Patients	Cost Per Claim avg	Total Claims
732,612,514	54,115	11,010	66,253

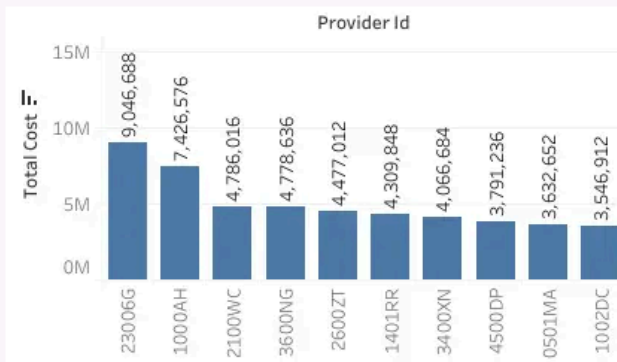
Patients per Provider



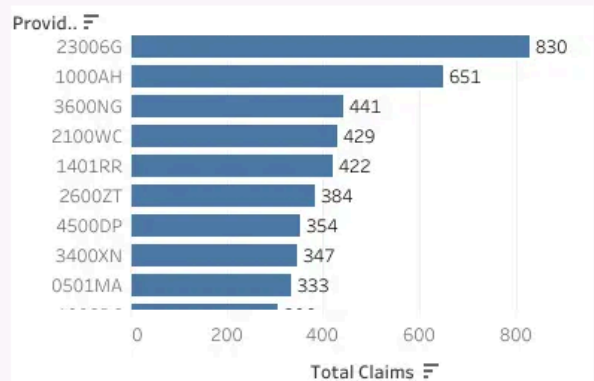
Average cost per claim



Total Claim cost by Provider



Total Claims handled by each provider



This dashboard provides a comprehensive analysis of healthcare provider performance using claims, patient and cost data. It helps us identify high cost providers, claim distribution patterns and efficiency across providers.

Total Healthcare Cost: \$732.6M

Total Patients: 54.1K

Average Cost per Claim: \$11.0K

Total Claims Processed: 66.3K

Providers with higher claim volumes exhibit more stable average claim costs.

Low volume providers show instability or inconsistency in average cost per claim.

- Patients per Provider -> Identifies providers handling highest patient volumes.
- Average cost per claim analysis -> Examines relationship between claim volume and average claim cost.
- Total claim cost by provider -> Highlights providers that are contributing most to healthcare.
- Claims handled by each provider -> Shows provider workload and claim distribution.